

NARAD Voting System - Complete Build Setup Guide

A comprehensive guide to set up the NARAD secure voting system with blockchain integration, WebAssembly encryption, and full-stack components.



Prerequisites

System Requirements

- **OS:** Ubuntu 22.04 LTS (tested and recommended)
- **RAM:** Minimum 8GB (16GB recommended for development)
- **Storage:** 15GB free space
- **Network:** Stable internet connection

Required Tools Overview

- Git with submodule support
 - Docker & Docker Compose
 - Node.js 18+ & npm
 - Rust 1.78.0 & Cargo
 - Solana CLI & Anchor Framework
 - CMake 3.15+ & cmake-js
 - Emscripten (for WebAssembly)
 - PostgreSQL (via Docker)
-



1. Initial Repository Setup

Clone the Repository

```
# Clone with all submodules
git clone --recursive https://github.com/raj-gigt/votingSys.git
cd votingSys

# Verify submodules are properly initialized
git submodule status
```

Expected submodule status:

```
30ef79ed937ca0fc7592ff73d162398773c6a5aa deps/curl (rc-8_14_0-2-37-
g30ef79ed9)
e823b0c34cea291bdb94d672731e1c1f08525557 deps/libtommath (v1.3.0-410-
ge823b0c)
```

Initialize Submodules (if needed)

```
# If submodules weren't auto-loaded
git submodule update --init --recursive

# Force update if having issues
git submodule foreach --recursive git reset --hard
git submodule update --init --recursive
```



2. Development Environment Setup

Install Node.js and Package Managers

```
# Install Node.js 18.x LTS
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt-get install -y nodejs

# Verify installation
node --version # Should be v18.x.x
npm --version  # Should be 9.x.x or higher
```

Install Rust and Cargo

```
# Install Rust via rustup
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source ~/.cargo/env

# Install and set specific Rust version for compatibility
rustup install 1.78.0
rustup default 1.78.0

# Verify installation
rustc --version # Should be 1.78.0
cargo --version # Should be 1.78.0
```

Install Solana CLI Tools

```
# Install Solana CLI version 2.0.24
sh -c "$(curl -sSfL https://release.solana.com/v2.0.24/install)"

# Add to PATH (add to ~/.bashrc or ~/.zshrc for persistence)
export PATH="$HOME/.local/share/solana/install/active_release/bin:$PATH"

# Verify installation
solana --version # Should be 2.0.24 (src:4c817c28; feat:607245837,
client:Agave)

# Set to localnet for development
solana config set --url localhost
```

Install Anchor Framework

```
# Install Anchor CLI version 0.30.1 (specific version for compatibility)
cargo install --git https://github.com/coral-xyz/anchor avm --locked --
force
avm install 0.30.1
avm use 0.30.1

# Verify installation
anchor --version # Should be 0.30.1
```

Install CMake and Build Tools

```
# Install CMake and build essentials
sudo apt-get update
sudo apt-get install -y cmake build-essential pkg-config

# Install cmake-js globally for Node.js native modules
npm install -g cmake-js

# Verify installation
cmake --version # Should be 3.15+
cmake-js --version
```

Install Emscripten (for WebAssembly)

```
# Clone Emscripten SDK in a separate directory (outside votingSys)
cd ~
git clone https://github.com/emscripten-core/emsdk.git
cd emsdk

# Download and install latest SDK tools
./emsdk install latest

# Make the "latest" SDK "active" for the current user
./emsdk activate latest

# Add to your shell profile for persistence
echo 'source ~/emsdk/emsdk_env.sh' >> ~/.bashrc
source ~/emsdk/emsdk_env.sh

# Verify Emscripten is active
emcc --version # Should show emscripten version

# Return to project directory
cd ~/votingSys # or wherever you cloned the project
```



3. Project Dependencies Setup

Critical Cargo Setup (Required after Solana installation)

```
# In the votingSys directory
cd votingSys

# Clean previous builds and reset cargo state
cargo clean

# Set specific Rust toolchain for this project (REQUIRED)
rustup override set 1.78.0

# Update other Cargo dependencies
cargo update

# Update specific problematic dependency (REQUIRED)
cargo update -p proc-macro2 --precise 1.0.94
```

Fix Cargo.lock Version Compatibility

```
# In the votingSys directory
cd votingSys

# Clean previous builds and reset cargo state
cargo clean

# Set specific Rust toolchain for this project (REQUIRED)
rustup override set 1.78.0

# Fix Cargo.lock version compatibility issue
# If you encounter "lock file version 4 requires -Znext-lockfile-bump"
error:
sed -i 's/version = 4/version = 3/' Cargo.lock

# Alternative method if sed doesn't work:
# 1. Open Cargo.lock in a text editor
# 2. Change the first line from 'version = 4' to 'version = 3'
# 3. Save the file

# Update other Cargo dependencies
cargo update

# Update specific problematic dependency (REQUIRED)
cargo update -p proc-macro2 --precise 1.0.94

# Verify the fix by building
anchor build
```

Note: The Cargo.lock version 4 format is not compatible with older Rust versions. Changing it to version 3 ensures compatibility with Rust 1.78.0 and the Anchor framework version used in this project.

```
---

## 🔗 4. Blockchain Setup (Solana/Anchor)

### Setup Solana Wallet
```bash
Generate a new keypair for development
solana-keygen new --outfile ~/.config/solana/id.json

Or use existing wallet
solana-keygen recover 'prompt:' --outfile ~/.config/solana/id.json

Set the wallet as default
solana config set --keypair ~/.config/solana/id.json

Check wallet address
solana address
```

## Start Solana Test Validator

```
Open a new terminal and navigate to votingSys directory
cd votingSys

Start the test validator (keep this terminal open)
solana-test-validator

In another terminal, airdrop SOL for testing
solana airdrop 10 # Request 10 SOL for testing
```

## Build and Deploy Smart Contract

```
In votingSys directory (different terminal from validator)
cd votingSys

Build the Anchor project
anchor build

Deploy to local validator
anchor deploy
```

## Update Program IDs

After deployment, copy the Program ID from the deployment output and update it in these files:

### 1. `programs/voting-sys/src/lib.rs`

```
declare_id!("YOUR_PROGRAM_ID_HERE");
```

### 2. `Anchor.toml`

```
[programs.localnet]
voting_sys = "YOUR_PROGRAM_ID_HERE"
```

### 3. `target/idl/voting_sys.json` (auto-updated)

### 4. `target/types/voting_sys.ts` (auto-updated)

```
After updating Program IDs, rebuild and redeploy
anchor build
anchor deploy

Copy IDL and types to backend
cp target/idl/voting_sys.json testing-2.0/voting_sys.json
cp target/types/voting_sys.ts testing-2.0/src/types/voting_sys.ts
```

---

## 5. Database Setup (PostgreSQL)

---

### Run PostgreSQL with Docker

```
Create and run PostgreSQL container
docker run --name voting-postgres \
 -e POSTGRES_USER=postgres \
 -e POSTGRES_PASSWORD=yourpassword \
 -e POSTGRES_DB=postgres \
 -p 5432:5432 \
 -d postgres:15

Verify container is running
docker ps
```

## Configure Environment Variables

```
Create .env file in testing-2.0 directory
cd testing-2.0
cat > .env << EOF
Database Configuration
DATABASE_URL="postgresql://postgres:yourpassword@localhost:5432/postgres?
sslmode=disable"

JWT Configuration
JWT_SECRET="your-super-secret-jwt-key-here"

Server Configuration
#PORT=3000
#NODE_ENV=development

Blockchain Configuration
#SOLANA_RPC_URL="http://localhost:8899"
#PROGRAM_ID="YOUR_PROGRAM_ID_HERE"
EOF
```

---

## 6. Build Native Modules

---

### Build Aggregator Module

```
cd aggregator

Install Node.js dependencies
npm install

Compile native C++ module
cmake-js compile

Verify the module was built
ls build/Release/aggregator.node # Should exist
```

### Build Collector Module



```
cd ../collector

Install Node.js dependencies
npm install

Compile native C++ module
cmake-js compile

Verify the module was built
ls build/Release/collector.node # Should exist
```

## Build WebAssembly Module

```
Return to project root
cd ..

Create build directory for WASM
mkdir -p build-wasm
cd build-wasm

Configure with Emscripten (Debug build for development)
emcmake cmake ../frontend/src/wasm -DCMAKE_BUILD_TYPE=Debug

For production builds use:
emcmake cmake ../frontend/src/wasm -DCMAKE_BUILD_TYPE=Release

Build (adjust parallel jobs based on your CPU cores)
cmake --build . --config Debug --parallel $(nproc)

Copy generated files to portal assets
cp encryption.js ../portal/public/assets/
cp encryption.wasm ../portal/public/assets/

Verify files were copied
ls -la ../portal/public/assets/encryption.*
```

---

## 7. Application Setup and Launch

---

### Setup Backend (testing-2.0)

```
cd testing-2.0

Install dependencies
npm install

Initialize and migrate database
npx prisma migrate dev --name init

Generate Prisma client
npx prisma generate

Verify database tables (optional)
npx prisma studio # Opens web interface at http://localhost:5555
```

## Setup Frontend Portal

```
cd ../portal

Install dependencies
npm install

Build for development
npm run build # Optional, for production builds
```

---

## 8. Running the Complete System

---

### 1. Start Solana Test Validator (if not already running)

```
Terminal 1: Keep this running throughout development
cd votingSys
solana-test-validator
```

### 2. Start Backend Server

```
Terminal 2: Backend API server
cd testing-2.0

Or production mode
npm run start
```

### 3. Start Frontend Portal

```
Terminal 3: Frontend development server
cd portal
npm run dev
```

### 4. Access the Application

- **Frontend Portal:** <http://localhost:5173>
- **Backend API:** <http://localhost:3000>
- **Prisma Studio:** <http://localhost:5555> (if running)
- **Solana RPC:** <http://localhost:8899>

---

## 9. Development Commands

---

### Useful Development Commands

```
Check Solana validator status
solana cluster-version

Check wallet balance
solana balance

View program logs
solana logs YOUR_PROGRAM_ID

Restart services if needed
Stop validator: Ctrl+C in validator terminal
Restart: solana-test-validator --reset

Reset database (if needed)
cd testing-2.0
npx prisma migrate reset

Rebuild WASM (if changes made)
cd build-wasm
cmake --build . --config Debug --parallel $(nproc)
cp encryption.* ../portal/public/assets/
```

## Testing Commands

```
Run Anchor tests
anchor test

Run backend tests
cd testing-2.0
npm test

Run blockchain integration tests
npm run test:blockchain
```

---

## 10. Troubleshooting

---

### Common Issues and Solutions

#### Submodule Issues

```
Reset submodules if corrupted
git submodule deinit --all -f
git submodule update --init --recursive
```

## Rust/Cargo Issues

```
Clean and rebuild
cargo clean
rustup override set 1.78.0
cargo update -p proc-macro2 --precise 1.0.94
anchor build
```

## Node Module Issues

```
Clear node_modules and reinstall
rm -rf node_modules package-lock.json
npm install
```

## Database Connection Issues

```
Check PostgreSQL container
docker ps
docker logs voting-postgres

Restart container if needed
docker restart voting-postgres
```

## WASM Build Issues

```
Ensure Emscripten is active
source ~/emsdk/emsdk_env.sh
emcc --version

Clean and rebuild
rm -rf build-wasm
mkdir build-wasm && cd build-wasm
emcmake cmake ../frontend/src/wasm -DCMAKE_BUILD_TYPE=Debug
cmake --build . --config Debug --parallel $(nproc)
```



## 11. Verification Checklist

---

Before running the system, verify:

- ☐ Solana test validator is running
  - ☐ PostgreSQL container is running
  - ☐ All native modules compiled successfully
  - ☐ WASM files are in portal/public/assets/
  - ☐ Environment variables are configured
  - ☐ Smart contract deployed successfully
  - ☐ Database migrations completed
  - ☐ All dependencies installed
- 



## 12. Production Considerations

---

### Security Notes

- Change default passwords in production
- Use environment-specific configuration
- Enable SSL/TLS for production deployments
- Secure private keys and mnemonics
- Use production Solana clusters (devnet/mainnet)

### Performance Optimization

- Use release builds for WASM modules
  - Enable production optimizations in frontend build
  - Configure proper database connection pooling
  - Set up proper logging and monitoring
- 



## 13. Additional Resources

---

- [Solana Documentation](#)
- [Anchor Framework Guide](#)
- [Emscripten Documentation](#)
- [Prisma Documentation](#)
- [React + Vite Documentation](#)

---

## Support

---

For issues and support:

1. Check the troubleshooting section above
  2. Review component-specific README files
  3. Create an issue on the GitHub repository
  4. Ensure all prerequisites are properly installed
- 

**Note:** This setup guide is specifically tested on Ubuntu 22.04. For other operating systems, commands may need to be adapted accordingly.